Image Classification Model using Transfer Learning in PyTorch

Overview

Transfer Learning is a machine learning algorithm where we reuse a pre-trained model as the starting point for a model on a new task. The intuition behind transfer learning for image classification is that if a model is trained on a large and general enough dataset, this model will effectively serve as a generic model of the visual world.

In this project, the ResNet model has been used as a pre-trained model for image classification in PyTorch. The images have been classified into classes of social security cards, driving licenses, and others. If you haven't visited already, here is the previous project of the series PyTorch Project to Build a LSTM Text Classification Model

Aim

- To understand the transfer learning and ResNet model.
- To build a transfer learning model for image classification in PyTorch

Tech Stack

➔ Language: Python

➔ Libraries: pytorch, pandas, matplotlib, numpy, opnecv_python_headless, torchvision

Data Description

Dataset used in this project are images of driving licenses, social security, and others categorized into respective categories. The images are of different shapes and sizes which are preprocessed before modeling.

Approach

- Data Loading
- Data Preprocessing
    - Resizing and scaling the images
    - Encoding the class labels

- Model Building and Training
    - ResNet model building in PyTorch

Modular Code Overview

```
Input
  |_Data
      |_Training_data
      |_Testing_data

MLPipeline
  |_CreateDataset.py
  |_ResNet_Model.py
  |_Train.py

Notebook
  |_TransferLearning_CNN.ipynb

Output
  |_model.pt

Engine.py
Readme.md
requirements.txt
```

Once you unzip the pytorch_resnet.zip file, you can find the following folders within it.

1. Input
2. ML_Pipeline
3. Notebook
4. Output
5. Engine.py
6. Readme.md
7. requirements.txt

1. The Input folder contains the data that we have for analysis. In our case, it contains training data and testing of images to be classified

2. The Notebook folder contains the jupyter notebook file of the project

3. The ML_pipeline is a folder that contains all the functions put into different python files, which are appropriately named. These python functions are then called inside the Engine.py file

4. The Output folder contains the saved model

5. The requirements.txt file has all the required libraries with respective versions. Kindly install the file by using the command **pip install -r requirements.txt**

6. **All the instructions for running the code are present in Readme.md file**

Takeaways

1. What is PyTorch?
2. PyTorch vs Tensorflow
3. What is transfer learning?
4. Why use transfer learning?
5. What is computer vision?
6. What is CNN?
7. Use of CNN
8. Architecture of CNN
9. How to use CNN with transfer learning?
10. What is the ResNet model?
11. The architecture of the ResNet model
12. Difference between VGG16 and ResNet
13. Data loading in PyTorch
14. Data Preprocessing in PyTorch
15. ResNet model building in PyTorch